REPORT

Section A

1)

a) Correlation refers to the degree to which the individual trees in the Random Forests are similar or exhibit a high level of agreement in their predictions.
Diversity, on the other hand, refers to the differences in the individual trees in the model. Too much correlation among the trees can lead to overfitting, as trees tend to make similar predictions for the input data. If all trees make similar errors on the same data points, the model will not be able to correct those errors effectively.

b) Too much diversity can result in a model where individual trees are making random predictions so that the overall performance may suffer and the model would be less robust.

c)

b) The curse of dimensionality becomes an issue when the data has high-dimensional spaces, i.e. a large number of features in the dataset. This is because data becomes sparse, and many feature combinations have little or no instances. The computational complexity of storing conditional probabilities grows exponentially as the number of features increases, and the model also becomes more prone to overfitting. The ways to mitigate this are:
   a. Feature selection: Select relevant features and discard the redundant features.
   b. Feature engineering: Create new features by merging/combining the existing features.
   c. Dimensionality reduction: By applying PCA or t-SNE, the dimension of the data can be reduced.
   d. Laplace smoothing: To handle sparse data, Laplace smoothing can be used if feature combinations are missing.

c) When a Naive Bayes classifier encounters a value of an attribute that was not present in the training dataset, it can face certain challenges. These challenges are:
   a. Zero Probability: If a feature value is absent in the training data, the conditional probability for that value will be zero. This can lead to issues during classification, as the posterior probability

for a class that depends on such a feature value will also be zero so the whole expression will become 0.

b. Misclassification: The absence of a feature value in the training data can lead to misclassification, as the model will have no information on how to handle this new, unseen value.

Yes this would affect the result as the one of the probabilities would become 0 which would lead to entire expression of naïve bayes=0 which would lead to misclassification.

To mitigate this following methods can be used:

Let c: number of classes

p:prior probability

m:parameter

i) Laplace smoothing:

$$P(A_i|C) = \frac{(N_{ic} + 1)}{(N_c + 1)}$$

ii) M-estimate

$$P(A_i|C) = \frac{(N_{ic} + mp)}{(N_c + m)}$$

Example:

X	Y	Target
0	0	0
0	1	0
1	0	1

Test (X=1,Y=1)

P(test=1) = P(target=1)*P(X=1|target=1)*P(Y=1|target=1) =1/3 * 1 * 0=0

P(test=0)= P(target=0)*P(X=1|target=0)*P(Y=1|target=0)=2/3 * 0 * ½=0

This gave a wrong classification

With Laplace smoothing

P(test=1) = P(target=1)*P(X=1|target=1)*P(Y=1|target=1)=2/4 * 1 * 1/2=1/4
P(test=0)= P(target=0)*P(X=1|target=0)*P(Y=1|target=0)=3/4 * 1/3 * 2/3=1/6

P(test=1)=P(test=1)/(P(test=1)+P(test=0)) =0.6

The prediction for the test is 1.

d) The Information Gain criterion used in decision tree algorithms can be biased when some attributes have more cardinality (distinct values) than others, and this can lead to a potential issue called the attribute bias problem. When an attribute has a higher cardinality, it tends to have a larger number of possible splits, and this can make it more likely to be selected as the root of the decision tree because it has greater information gain

This bias can be problematic because it might cause the decision tree to overfit the training data.
To mitigate this problem other measure can be used such as:

- Gini Impurity:. In each split, the attribute with the lowest Gini Impurity is chosen as the root node. Gini Impurity does not favor attributes with high cardinality and can provide a more balanced tree.

$$Gini(X) = 1 - \sum_{i=1}^{c} p_i^2$$

- Gain Ratio:. It divides the Information Gain by the intrinsic information of an attribute, which takes into account the number of branches created by the attribute. This discourages the selection of attributes with high cardinality, as the intrinsic information increases with the number of branches.

Example

Feature 1	Feature 2	target
A	X	1
A	Y	0
B	X	0
B	Y	1
C	X	1

Information Gain for "Feature 1":

Entropy before split (S) = -3/5 * log2(3/5) - 2/5 * log2(2/5) ≈ 0.971

Entropy after splitting on "Feature 1":

Entropy(A) = -1/2 * log2(1/2) - 1/2 * log2(1/2) = 1

Entropy(B) = -1/2 * log2(1/2) - 1/2 * log2(1/2) = 1

Entropy(C) = 0

Entropy(F1) = (2/5) * 1 + (2/5) * 1 + (1/5) * 0 = 0.8

Information Gain(F1) = S - S_F1 = 0.971 - 0.8 ≈ 0.171

Information Gain for "Feature 2":

Entropy before split (S) = 0.971

Entropy after splitting on "Feature 2":

Entropy(X) = -2/3 * log2(2/3) - 1/3 * log2(1/3) ≈ 0.918

Entropy(Y) = -1/2 * log2(1/2) - 1/2 * log2(1/2) = 1

Entropy(F2) = (3/5) * 0.918 + (2/5) * 1 =0.961

Information Gain(F2) = S - S_F2 ≈ 0.971 - 0.961= 0.010

Gini Impurity for "Feature 1":

Gini Impurity before split (Gini(S)) = 1 - (3/5)^2 - (2/5)^2 = 0.48

Gini Impurity after splitting on "Feature 1":

Gini(A) = 1 - (1/2)^2 - (1/2)^2 = 0.5

Gini(B) = 1 - (1/2)^2 - (1/2)^2 = 0.5

Gini(C) = 0

Gini(F1)= (2/5) * 0.5 + (2/5) * 0.5 + (1/5) * 0 = 0.4

Gini Gain(F1) = Gini(S) - Gini(F1) = 0.48 - 0.4 = 0.08

Gini Impurity for "Feature 2":

Gini Impurity before split (Gini(S)) = 0.48

Gini Impurity after splitting on "Feature 2":

Gini(X) = 1 - (2/3)^2 - (1/3)^2 ≈ 0.444

Gini(Y) = 1 - (1/2)^2 - (1/2)^2 = 0.5

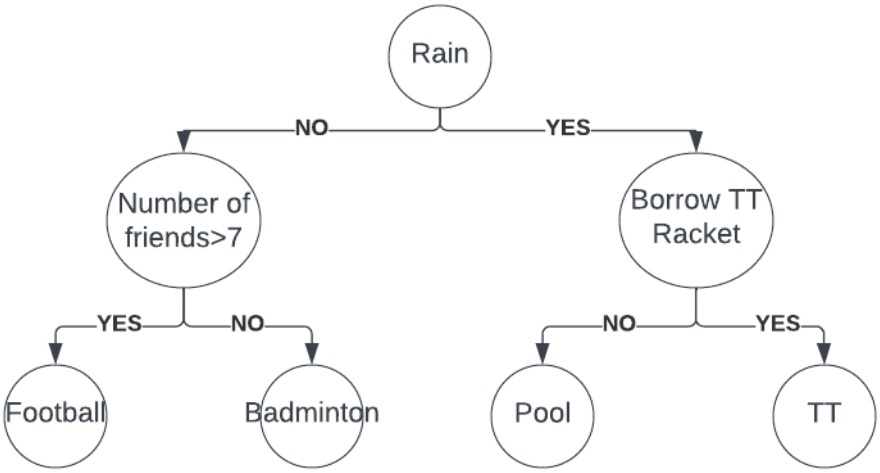
Gini(F2) = (3/5) * 0.444 + (2/5) * 0.5 = 0.47

Gini Gain(F2)= 0.48-0.47=0.01

In this example, Information Gain for "Feature 1" is 0.171 more distriminating power and Information Gain for "Feature 2" is only 0.010, suggesting that it provides less discrimination power. On the other hand, Gini Gain for "Feature 1" is 0.08, while Gini Gain for "Feature 2" is 0.01. This demonstrates that Gini Impurity is not as biased by attribute cardinality, and it provides more balanced results in this scenario.

2)

a)



P(Football=yes)= P(No of friends>7=yes|Rain=no)* P(rain=no)

P(Badminton=yes)=P(No of friends>7=no|Rain=no)*P(rain=no)

P(Pool=yes)= P(Borrow tt racket=no|Rain=yes)*P(rain=yes)

P(TT=yes)= P(Borrow tt racket=yes|Rain=yes)*P(rain=yes)

b)

Given :

P(app predict rain=yes)= 0.3

P(app predict clear=yes)= 0.7

P(app predict rain=yes | Rainy=yes)= 0.8

P(app predict clear=yes | Clear=yes) = 0.9

P(rainy=yes | app predict rainy=yes)=?

Using bayes theorem

P(rainy=yes | app predict rainy=yes)= P(app predict rain=yes |rainy=yes) * P(rainy=yes)/P(app predict rain=yes)

P(app predict rain=yes |rainy= yes) = 0.8

P(app predict rain=yes) =0.3

P(rainy)=?

Using total probability theorem

P(app predict rain=yes) = P(app predict rain=yes| rainy=yes) *

P(rainy=yes) + P(app predict rain=yes| clear= yes)*P(clear =yes)

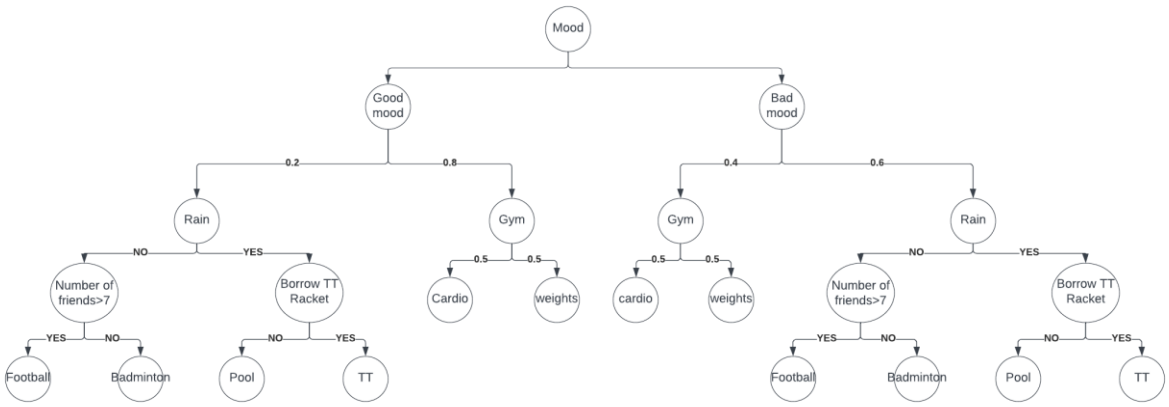
P(app predict rain=yes) = 0.8 * P(rainy=yes) + 0.1(1-P(rainy=yes))

0.3=0.8 * P(rainy=yes) + 0.1(1-P(rainy=yes))

P(rainy=yes)=2/7

P(rainy=yes | app predict rainy=yes)= 0.8 *2 /(0.3*7)= 0.7619

c)



P(Football)= P(number of friends>7=yes, rain=no, original plan, good mood) +
P(number of friends>7 =yes, rain=no, original plan, bad mood)

P(Badminton)= P(number of friends>7 =no, rain=no, original plan, good mood) + P(number of friends>7 =no, rain=no, original plan, bad mood)

=P(number of friends>7 =no| rain=no), original plan, good mood) + P(number of friends>7 =no, rain=no, original plan, bad mood)

P(Pool)= P(borrow tt racket=no, rain=yes, original plan, good mood) + P(borrow tt racket=no, rain=yes ,original plan, bad mood)

P(TT)= P(borrow tt racket=yes, rain=yes, original plan, good

mood) + P(borrow tt racket=yes, rain=yes, original plan, bad mood)

P(cardio)=P(cardio, gym, good mood)+P(cardio, gym, bad mood)

P(weights)= P(weights, gym, good mood)+P(weights, gym, bad mood)

d)

Given

P(good mood)=0.6

P(bad mood)= 0.4

F= amount of sleep

P(F = 7 | Good mood) = 0.7

P(F = 7 | Bad mood) = 0.45

Using bayes theorem

P(Good mood| F=7) = P(F=7|good mood)*P(good mood)/P(F=7)

P(F=7)=?

Using total probability

P(F=7) = P(F=7|good mood)*P(good mood) + P(F=7| bad mood)* P(bad mood)

P(F=7)= 0.7 * 0.6+0.45*0.4=0.6

P(Good mood | F=7) = 0.7 * 0.6/0.6 =0.7

P(bad mood|F=7) = 0.3

Possibilities:

P(original plan|F=7)= P(original plan=yes , good mood= yes |F=7) +

P(original plan=yes , good mood= no|F=7)

P(original plan|F=7) = 0.7 *0.2+0.3*0.6 = 0.32

P(cardio|F=7) =P(cardio, gym=yes, good mood=yes|F=7) +

P(cardio, gym=yes, mood=yes |F=7)

P(cardio|F=7)= 0.7*0.8*0.5 + 0.3*0.4*0.5 = 0.34

P(weight|F=7) =P(weight, gym=yes, good mood=yes|F=7) +

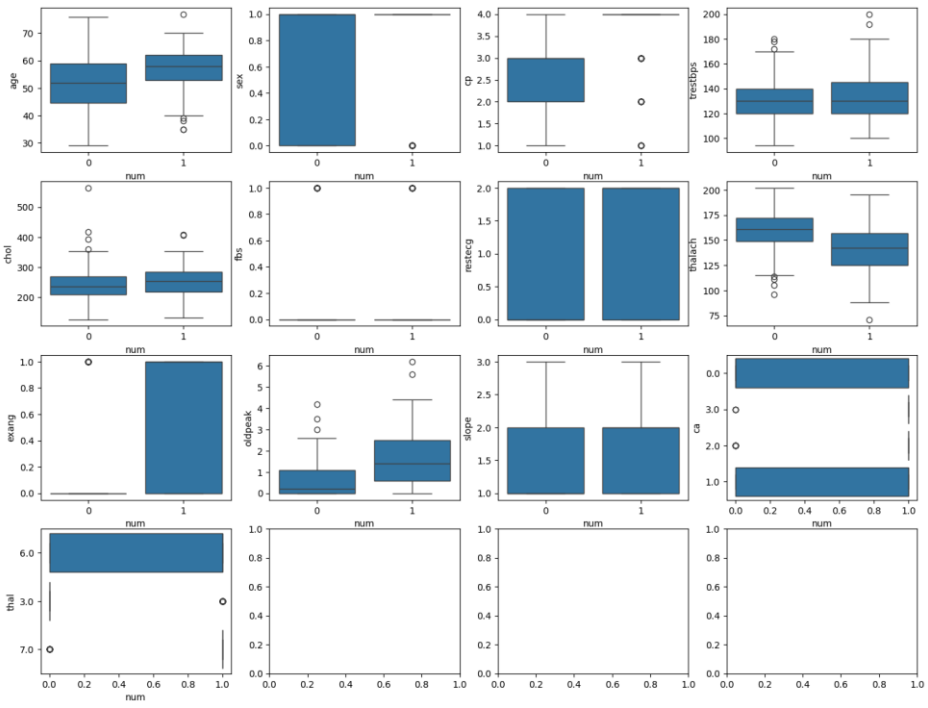
P(weight, gym=yes, good mood=yes|F=7)

P(weight | F=7)= 0.7*0.8*0.5 + 0.3*0.4*0.5 = 0.34

Section B

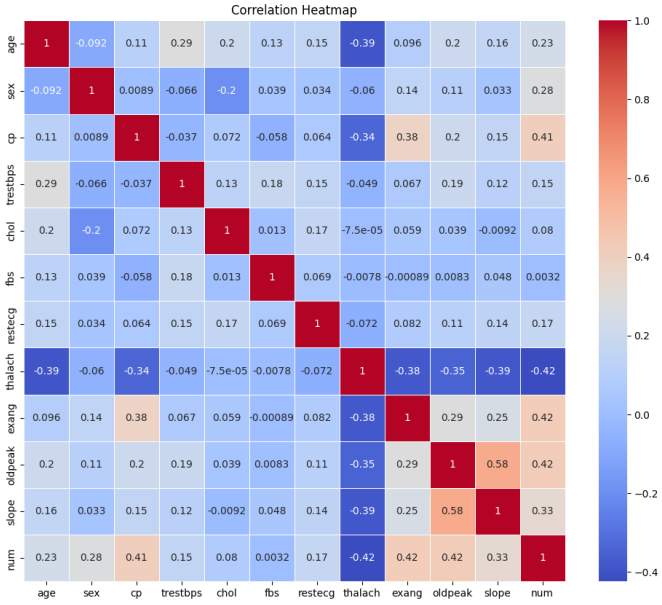
3)

Box plots:



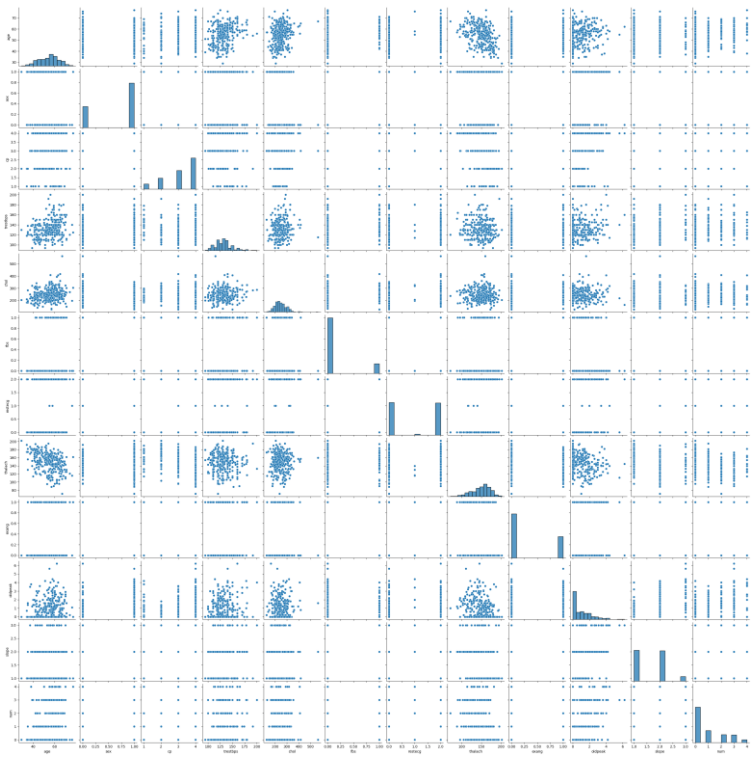
- The cholesterol and thalach has some outliers which could be removed.

Correlation heatmap

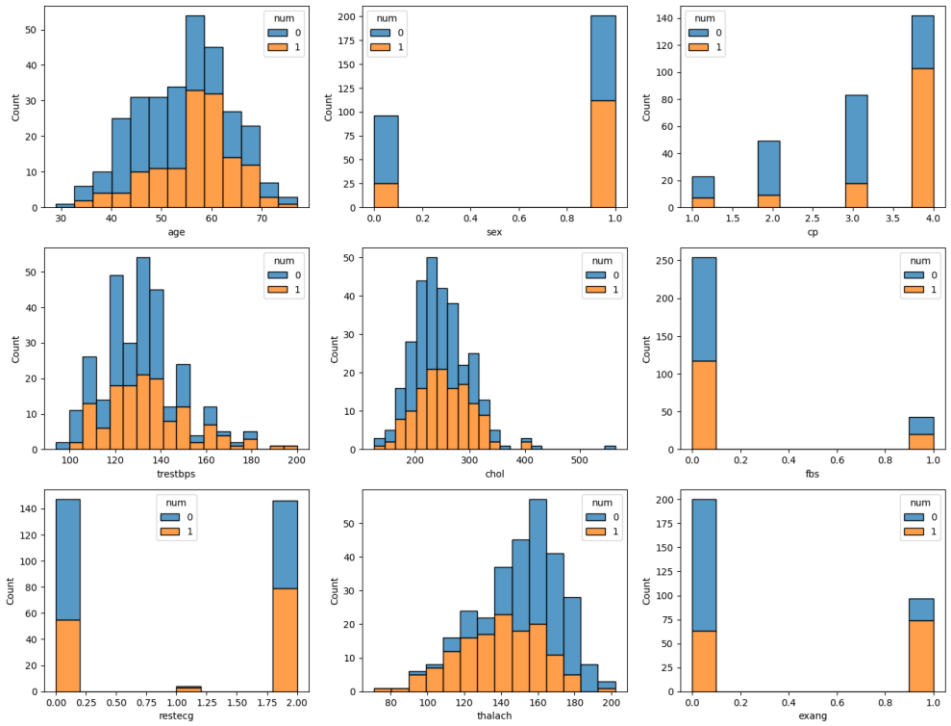


- Slope and oldpeak have considerable correlation with each other
- Num is moderately correlated to cp, exang, oldpeak and slope which means these features are more important

Pair plot



Histogram



4)

```
Best criterion for attribute selection: gini
Accuracy with the best criterion: 0.77
```

5)

```
DecisionTreeClassifier(random_state=20)
gini
[None, 6, 7, 8, 9, 10, 11, 12, 13]
Fitting 5 folds for each of 2664 candidates, totalling 13320 fits
best min samples split: 22
best max features: 7
DecisionTreeClassifier(random_state=20)
Accuracy with the best criterion: 0.78
```

6)

```
Fitting 5 folds for each of 320 candidates, totalling 1600 fits
Best hyperparameters:
n_estimators: 100
max_depth: 30
min_samples_split: 20

Classification Report:
              precision    recall  f1-score   support

           0       0.75      0.90      0.82        30
           1       0.88      0.70      0.78        30

    accuracy                           0.80        60
   macro avg       0.81      0.80      0.80        60
weighted avg       0.81      0.80      0.80        60

Precision for class 0: 0.88
Recall for class 0: 0.70
F1-Score for class 0: 0.78
Accuracy: 0.80
```

Section C

gini_impurity(self, y): Calculate Gini impurity of a node

max_depth(self, depth): Function to set the maximum depth of the tree

cost_function(self, y, left_y, right_y) :Calculate Gini Gain

split_data(self, X, y, feature, value)(helper function): Split data into left and right based on a feature and value

find_best_split(self, X, y)(helper function):

make_split(self, X, y, depth=0): build tree

fit(self, X, y)

predict_instance(self, instance, node)(helper function): check if the node is a leaf node

predict(self, X):

score(self, X, y):

```
Accuracy on train: 0.9953292853806632
Accuracy on test: 0.9869402985074627
```